

Machine Learning Engineer, Practical Exercise

Senior ML Engineer role · Take-home practical

Time: we expect a strong submission to take around **3 focused hours**, treat that as a *budget, not a rule*. How you balance scope against time is part of what we're assessing, so build what you'd confidently ship in that window and tell us what you traded off. **Deadline:** within 48 hours of receipt (schedule the time whenever suits you).

Submit: a GitHub repo link + a short video link, in a single reply.

AI tools: allowed. We only require that you can explain and defend every decision you ship.

The task, Support-ticket triage

You're given **~400 labeled customer-support messages** from a crypto/fintech product (`train.csv`). Each belongs to one of four routes: **account-access**, **transaction-dispute**, **fraud-report**, **general**. Build a service that classifies an incoming message into the correct route.

Scoring note: we evaluate on a **hidden holdout set**, so build something that generalizes, don't over-fit the data you can see.

What to build (required)

- **A classifier** for the four routes, a classical baseline (e.g., TF-IDF + logistic regression or gradient boosting) is perfectly sufficient.
- **A proper evaluation:** a train/validation split, a deliberate choice about class imbalance, and a justified metric.
- **Clean, tested Python:** at least two meaningful tests, basic input validation, and a simple `predict(text) → Label` interface (a function or CLI is fine; a small API is optional, not required).
- **A holdout-scoring entry point:** a script/command that reads a CSV of messages and writes out predictions.

Optional, only if you have budget left

These are **stretch items, not expectations**. We're more interested in a tight, well-reasoned core than a sprawling submission, deciding *not* to do these, and telling us why, is a perfectly strong (often the strongest) answer.

- An LLM-based classifier to compare against your baseline.
- A small API (e.g., FastAPI) around your `predict` function.
- Containerization, CI, or anything else you'd ship for real.

A short "Scope & trade-offs" note

In your README *or* in the video, briefly tell us:

- What you **prioritized**, and what you **deliberately left out**.
- What you'd do with more time.
- Roughly how long you actually spent (honest estimate is fine).

Deliver via GitHub

- A repo (public, or invite us) with **real commit history**, please commit as you work, not in one final push.
- A **README** with clear run instructions.

Plus a short video (≤ 8 min)

Voice required; screen-share optional. Walk us through your solution and answer these out loud:

1. Why did you choose your evaluation metric, given that **fraud-report** is the highest-stakes route to get wrong?
2. How did you handle class imbalance, and how would you know if it were hurting you?
3. Point to **one decision in your own code** you were unsure about, and explain the trade-off you made.
4. If this scaled to **10,000 requests/minute**, or if you swapped your model for an **LLM**, when would each approach be the right call?

How we evaluate

- **ML mechanics & metric reasoning**, do you understand *why*, not just *what* (metric choice, imbalance, data leakage).
- **Prioritization & scoping**, sensible time-vs-features trade-offs, clearly reasoned.
- **Judgment**, choosing the right tool for the problem, not maximum complexity.
- **Code quality & testing**, readable, structured, genuinely tested.
- **Communication**, clear, confident explanation of your decisions.

To submit: reply with (1) your GitHub repo link and (2) your video link. That's it. Thanks, we're looking forward to seeing how you think.

Top candidates from this exercise are invited to a 45-minute live session, which builds directly on the work you submit here.